山东大学 计算机科学与技术 学院

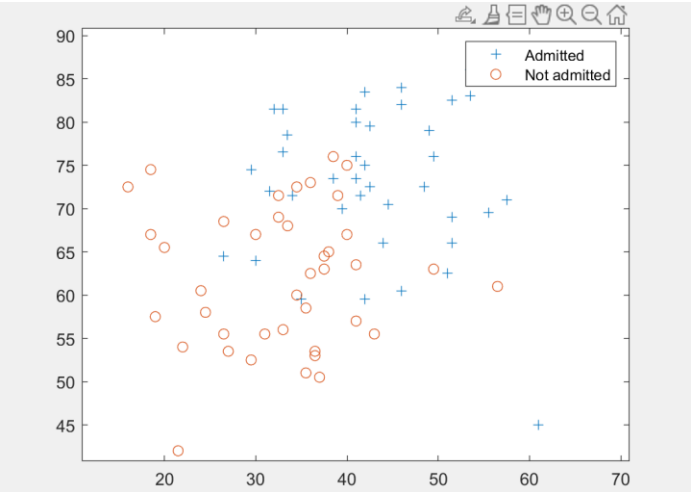
机器学习 课程实验报告

学号：202200130077	姓名： 薛润泽	邮箱：
实验题目：Experiment 4 : Logistic Regression and Newton's Method		

实验过程：

（记录实验过程、遇到的问题和实验结果。可以适当配以关键代码辅助说明，但不要大段贴代码。）

1、将 x0 = 1 添加到 x 矩阵中后，将两次测试的同学通过录取和被录取分类。并将第一次考试成绩为 x 轴，第二次考试成绩为 y 轴，将两类点可视化表示出，结果如下图：



2、实验给出了逻辑回归的假设函数

$$h_\theta(x) = g(\theta^T x) = \frac{1}{1+e^{-\theta^T x}} = P(y=1 \mid x;\theta)$$

我们通过该假设函数来进行对打分的概率映射。可以看到，当$-\theta^T x$值为 0 时，$h_\theta x$值为 0.5，当$h_\theta x$值<0.5 时，我们认为是 Not admitted，当$h_\theta x$值>0.5 时，我们认为是 Admitted。对应到$-\theta^T x$，也就是打分值为正值和负值的情况。

逻辑回归损失函数的一阶导数如下：

$$\nabla_\theta J = \frac{1}{m}\sum_{i=1}^{m}(h_\theta(x^{(i)}) - y^{(i)})x^{(i)}$$

其二阶导数如下：

$$\boldsymbol{H} = \sum_{i=1}^{m} \boldsymbol{x_i} z\left(\boldsymbol{w^T x_i}\right) z\left(-\boldsymbol{w^T x_i}\right) \boldsymbol{x_i^T}$$

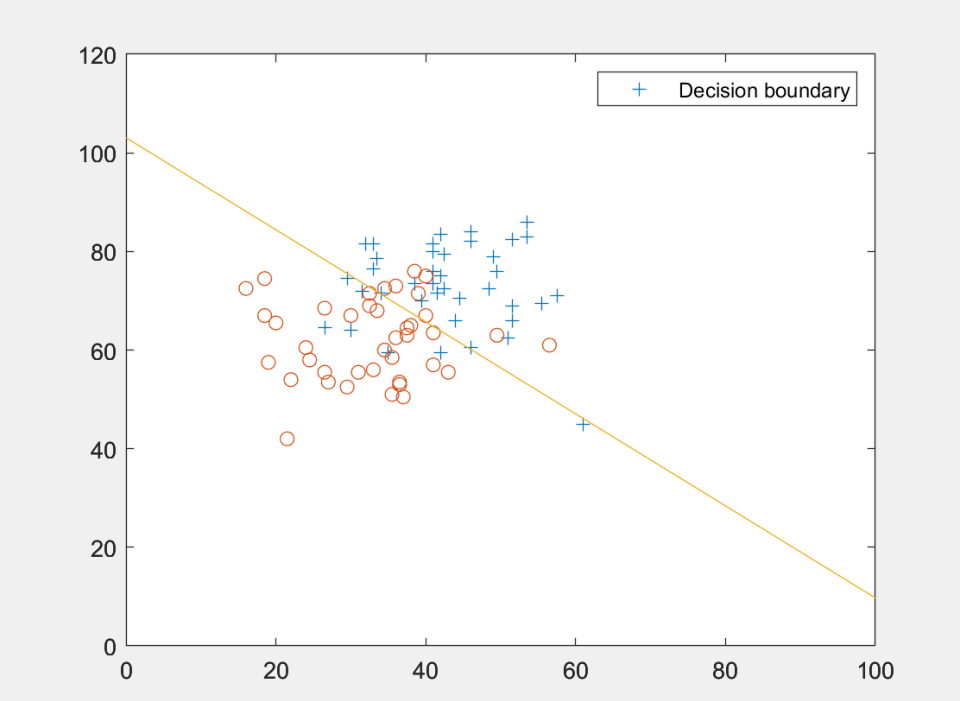
所以，按照牛顿法迭代公式：

$$\theta^{(t+1)} = \theta^{(t)} - H^{-1}\nabla_\theta J$$

我们将 θ 整体带入到迭代公式中，进行整体迭代，最终得到收敛的 θ 值。
使用得到的 θ，计算$\theta^T x = 0$，整理得公式：

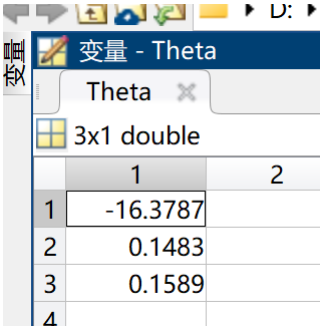
$$x_3 = \frac{-(\theta_1 + \theta_2 x_2)}{\theta_3}$$

然后画图：



回答问题：

1、最后的 Theta = [−16.3787, 0.1483, 0.1589]；5 次迭代后会收敛



2、被录取的概率是 0.3320, 不被录取概率是 1−0.3320 = 0.6680

```
22
23      testx = [1,20,80];
24      predict = hx(Theta, testx')
25
```

命令行窗口

```
>> ex4main
>> ex4main

predict =

    0.3320
```

代码附录：

%ex4main.m

```
load("ex4x.dat");
load("ex4y.dat");
[m,~] = size(ex4x);
ex4x = [ones(m,1),ex4x];
pos = find(ex4y == 1);
neg = find(ex4y == 0);
plot(ex4x(pos,2), ex4x(pos,3),'+'); hold on;
plot(ex4x(neg,2),ex4x(neg,3),'o')
legend("Admitted","Not admitted");
hold on;
%
Theta = [0; 0; 0];

for i = 1 : 5
Theta =  Theta - inv(hessian(m , Theta, ex4x))*Jd1(m,Theta,ex4x, ex4y);
end

x2 =0: 0.1:100;
y2 = -(Theta(1) + Theta(2)*x2)/Theta(3);
plot(x2, y2);
legend("Decision boundary");

testx = [1,20,80];
predict = hx(Theta, testx')
```

%Jd1.m

```
function [res] = Jd1(m, Theta, ex4x, ex4y)
    res = 0;
    for i = 1:m
        res  = res + (hx(Theta, ex4x(i,:)') - ex4y(i))*ex4x(i,:)';
```

```
    end
    res = res/m;
end

%hx.m
function [g] = hx(Theta, ex4x)
g = 1.0/(1.0 + exp(-Theta'*ex4x));
end

%hessian.m

function [res] = hessian(m, Theta, ex4x)

    res = [0 0 0; 0 0 0;0 0 0];
    for i = 1:m
    res = res + ex4x(i,:)'*hx(Theta, ex4x(i,:)') * hx(-Theta,
ex4x(i,:)')*ex4x(i,:);
    end
    res = res/m;
end
```

结果分析与体会：

分析：结果正确合理。

体会：开始时，受到梯度下降法的影响，一直在纠结牛顿法更新 θ 是要一个一个来更新θ_i还是可以直接更新 θ 来更新。最后得出结论，可以直接更新 θ 。因为牛顿法的公式中使用的是损失函数的一阶导数除一个 Hessian 矩阵，得出的最后结果是一个列向量，所以要求 θ 以一个列向量的结果来更新。